

Министерство науки и высшего образования
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнили: ст. гр. 25BBB2
Потёмкин Д.С.
Гладков А.М.

Проверил: к.п.н., доцент
Юрова О.В.
Деев М.В.

Пенза 2026 г.

Название и цель работы

Название: Оценка времени выполнения программ.

Цель работы: изучение способов измерения времени выполнения программ на языке Си средствами библиотеки `time.h`; определение сложности алгоритма умножения матриц и сравнение времени работы сортировки Шелла, быстрой сортировки и библиотечной функции `qsort`.

Лабораторное задание

1. Определить порядок сложности программы умножения двух квадратных матриц. Измерить время полного вычисления и отдельно время умножения для размеров 100, 200, 400, 1000, 2000, 4000 и 10000. Построить график и сравнить его с теоретической оценкой.

2. Сравнить сортировку Шелла, быструю сортировку и `qsort` на случайном, возрастающем, убывающем массиве и на массиве, первая половина которого возрастает, а вторая убывает.

Пояснительный текст к программам

В работе рассматриваются программы на языке Си из методических указаний. Библиотека `stdio.h` используется для ввода-вывода, `stdlib.h` - для генерации случайных чисел и функции `qsort`, `time.h` - для работы со временем.

Продолжительность вычисления определяется по разности показаний `clock()` до и после исследуемого участка. Деление выполняется в вещественном типе:

$$t = (\text{double})(\text{end} - \text{start}) / \text{CLOCKS_PER_SEC}$$

Исходный код и экспериментальные данные

Листинги ниже воспроизведены из файла «Лабораторная работа №2.doc» без исправления ошибок. Таблицы и графики взяты из ранее выполненной экспериментальной серии исправленной измерительной версии. Они не являются результатами запуска приведённых листингов без изменений.

Измерения выполнены в Windows на AMD Ryzen 7 5700G, GCC 15.2.0, оптимизация -O2, один поток. Для сортировок использован массив из 10000 целых чисел. Замеры не относятся к сборке MSVC в Visual Studio.

Пояснения к заданию 1

Листинг 1. Умножение матриц из методических указаний.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    clock_t start, end; // объявляем переменные для определения времени выполнения

    int i=0, j=0, r;
    int a[200][200], b[200][200], c[200][200], elem_c;

    srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    while(i<200)
    {
        while(j<200)
        {
            a[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
            j++;
        }
        i++;
    }
    srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    i=0; j=0;
    while(i<200)
    {
        while(j<200)
        {
            b[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
            j++;
        }
        i++;
    }

    for(i=0;i<200;i++)
    {
        for(j=0;j<200;j++)
        {
            elem_c=0;
            for(r=0;r<200;r++)
            {
                elem_c=elem_c+a[i][r]*b[r][j];
                c[i][j]=elem_c;
            }
        }
    }

    return(0);
}
```

Переменные и основные операции

a[200][200], **b[200][200]** - исходные матрицы; **c[200][200]** - матрица результата; **i**, **j**, **r** - индексы циклов; **elem_c** - накопленная сумма для одного элемента результата; **start**, **end** - объявленные переменные типа `clock_t`.

Функция `main` содержит заполнение матриц и три вложенных цикла умножения. Для каждой строки *i* первой матрицы и каждого столбца *j* второй матрицы переменная *r* перебирает пары элементов. Их произведения суммируются в `elem_c`.

Математическая формула вычисления элемента результата:

$$c[i][j] = a[i][0] \cdot b[0][j] + \dots + a[i][n-1] \cdot b[n-1][j].$$

Оценка сложности

При обобщении размеров матриц до $n \times n$ каждый из трёх циклов умножения выполняется *n* раз. Число выполнений внутренней операции равно n^3 . Структура алгоритма имеет временную сложность $O(n^3)$, точнее $\Theta(n^3)$. Для хранения трёх матриц требуется $\Theta(n^2)$ памяти. Для фиксированного размера 200 задача имеет постоянный объём; асимптотическая оценка относится к изменяемому *n*.

В простой кубической модели увеличение стороны матрицы в два раза повышает время примерно в восемь раз. Для сравнения с измерениями использована функция $T(n) = T(1000) \cdot (n/1000)^3$, где $T(1000) = 0,623$ с.

Особенности исходного листинга

В циклах заполнения индекс *j* не обнуляется при переходе к следующей строке. Поэтому инициализируется только первая строка каждой матрицы, а последующее чтение остальных элементов приводит к неопределённому поведению. Переменные `start` и `end` объявлены, но вызовов `clock()` и вывода времени в листинге нет.

Повторный `srand(time(NULL))` может задать то же начальное состояние генератора. Присваивание `c[i][j]` расположено внутри цикла *r* и записывает промежуточные суммы. Эти особенности сохранены в приведённом коде.

В измерительной версии были исправлены заполнение матриц и работа с памятью, добавлены таймеры и проверки. Порядок циклов *i*, *j*, *r* сохранён; окончательная сумма записывалась в *c* после цикла *r*. Поэтому эксперимент характеризует этот вариант реализации алгоритма.

Пояснения к заданию 2

Листинг 2. Сортировка Шелла из методических указаний.

```
void shell(int *items, int count)
{
    int i, j, gap, k;
    int x, a[5];

    a[0]=9; a[1]=5; a[2]=3; a[3]=2; a[4]=1;

    for(k=0; k < 5; k++) {
        gap = a[k];
        for(i=gap; i < count; ++i) {
            x = items[i];
            for(j=i-gap; (x < items[j]) && (j >= 0); j=j-gap)
                items[j+gap] = items[j];
            items[j+gap] = x;
        }
    }
}
```

Листинг 3. Быстрая сортировка из методических указаний.

```
void qs(int *items, int left, int right) //вызов функции: qs(items, 0, count-1);
{
    int i, j;
    int x, y;

    i = left; j = right;

    /* выбор компаранда */
    x = items[(left+right)/2];

    do {
        while((items[i] < x) && (i < right)) i++;
        while((x < items[j]) && (j > left)) j--;

        if(i <= j) {
            y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++; j--;
        }
    } while(i <= j);

    if(left < j) qs(items, left, j);
    if(i < right) qs(items, i, right);
}
```

Листинги приведены без исправлений. В условии внутреннего цикла shell обращение к `items[j]` стоит до проверки $j \geq 0$; при отрицательном j возможно чтение за границей массива. В измерительной версии порядок этих проверок был изменён.

Переменные и функции сортировки

shell(items, count) упорядочивает массив items из count элементов. Массив a задаёт шаги 9, 5, 3, 2, 1; gap хранит текущий шаг, k выбирает его, i и j задают позиции, x временно хранит вставляемый элемент. На каждом шаге выполняется сортировка вставками элементов, расположенных на расстоянии gap.

qs(items, left, right) сортирует участок массива от left до right включительно. Переменная x хранит опорное значение из середины участка, i и j движутся навстречу друг другу, u используется при обмене. После разделения функция рекурсивно обрабатывает оставшиеся части.

qsort - стандартная функция сортировки из stdlib.h. Ей передаются адрес массива, число элементов, размер элемента и функция сравнения. Методичка не содержит её отдельного листинга. Стандарт Си не требует от qsort конкретного внутреннего алгоритма.

Сложность и влияние исходных данных

Для варианта Шелла с фиксированными шагами 9, 5, 3, 2, 1 лучший случай имеет сложность $\Theta(n)$, а худший - $\Theta(n^2)$. Для быстрой сортировки средняя сложность составляет $O(n \log n)$, худшая - $O(n^2)$. Выбор среднего индекса не гарантирует равномерное распределение значений.

Порядок измерения времени

Для каждого из четырёх типов входных данных всем алгоритмам передавались одинаковые копии исходного массива. Перед каждым повтором исходный порядок восстанавливался. Копирование и сравнение с эталоном выполнялись вне измеряемого интервала.

Короткие операции повторялись до накопления 0,2 с измеренного времени либо до 100000 повторов. В таблицах приведено среднее за один запуск. Длительные умножения матриц выполнялись один раз. Последние знаки среднего не означают соответствующую точность таймера.

Для матриц отдельно измерялось умножение и полный вычислительный блок: выделение памяти, заполнение, умножение, контрольная сумма, проверка трёх элементов и освобождение памяти. Вывод CSV и запуск процесса в полное время не входят.

Результаты работы программ

Задание 1. В таблице 1 приведены данные измерительной версии для всех размеров матриц из задания.

Таблица 1 - Время умножения матриц

Размер $n \times n$	Умножение, с	Полное вычисление, с
100×100	0,000377	0,000491
200×200	0,003045	0,003515
400×400	0,024889	0,026667
1000×1000	0,623000	0,636000
2000×2000	10,346000	10,402000
4000×4000	151,398000	151,611000
10000×10000	3370,812000	3372,171000

Умножение матриц — измерения и кубическая модель

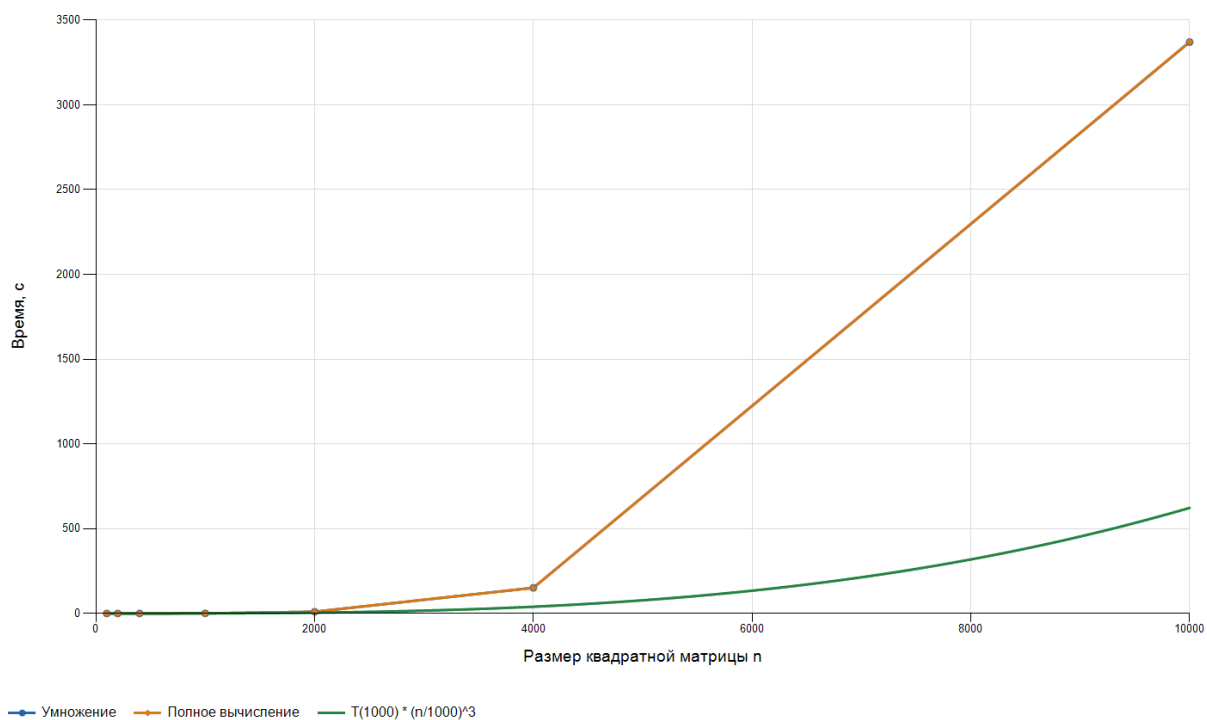


Рисунок 1 - Время вычислений и кубическая модель

При $n = 10000$ умножение заняло 3370,812 с, то есть около 56,18 мин. Полное вычисление заняло 3372,171 с. Основная доля времени приходится на перемножение матриц.

Сравнение измерений с теоретической оценкой

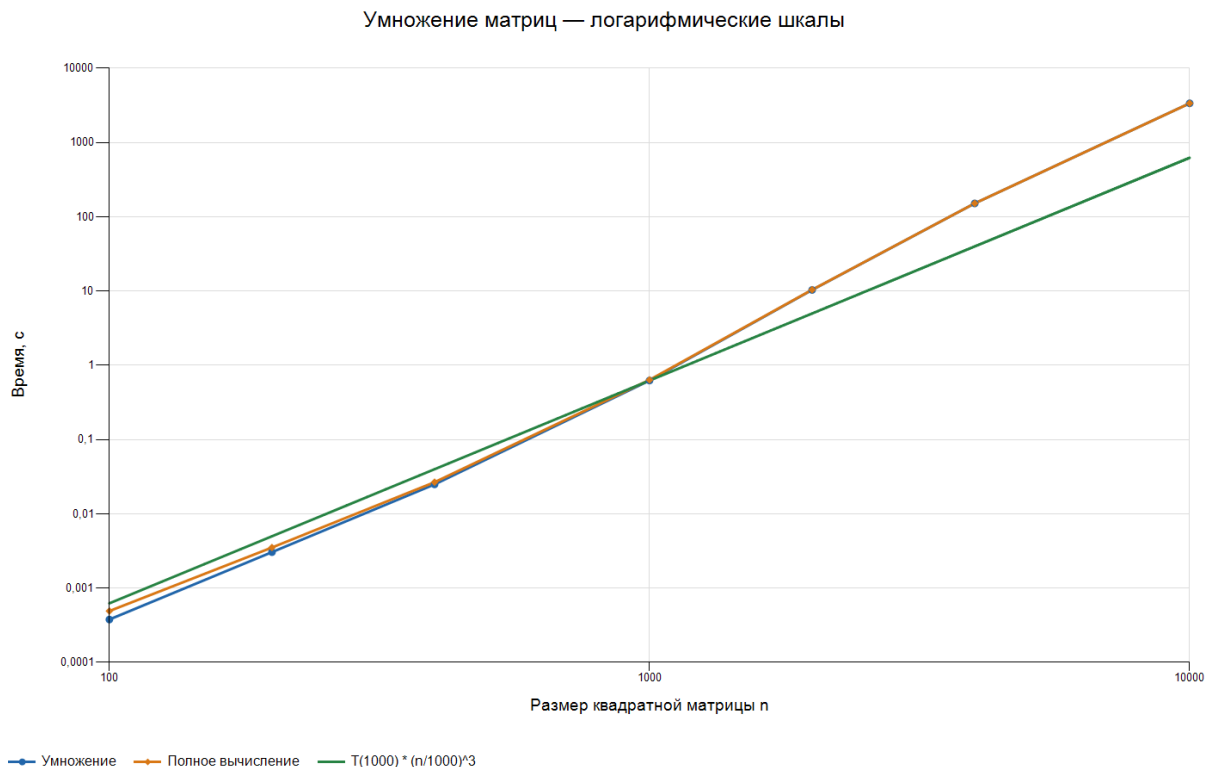


Рисунок 2 - Зависимость времени в логарифмических координатах

Логарифмические шкалы позволяют рассмотреть и малые, и большие размеры. Кубическая модель нормирована по размеру $n = 1000$. Для $n = 10000$ она даёт 623 с, тогда как измерение составляет 3370,812 с, примерно в 5,41 раза больше.

При увеличении размера от 1000 до 2000 время умножения выросло примерно в 16,61 раза, а от 2000 до 4000 - в 14,63 раза. Эти коэффициенты превышают восемь, предсказываемое моделью с постоянным временем одной операции.

Число операций алгоритма остаётся кубическим. Расхождение с простой моделью связано с тем, что стоимость обращения к памяти не постоянна для разных размеров: элементы второй матрицы читаются по столбцам с большим шагом. Влияние кэша, памяти и текущей нагрузки не позволяет выводить точное время только из О-оценки.

Поэтому график показывает рост времени и отклонение от модели с постоянным коэффициентом, но не служит доказательством точного закона $T(n) = C \cdot n^3$ на всём диапазоне.

Результаты сравнения сортировок

Задание 2. Размер каждого массива составляет 10000 элементов. Значения таблицы 2 выражены в миллисекундах.

Таблица 2 - Среднее время одной сортировки

Исходный порядок	Шелла, мс	Быстрая, мс	qsort, мс
Случайный	1,135593	0,402414	0,689655
Возрастающий	0,026642	0,051361	0,178571
Убывающий	1,932692	0,054451	0,195503
Первая половина возрастает, вторая убывает	0,990099	8,541667	0,746269

Сравнение сортировок — 10000 целых чисел

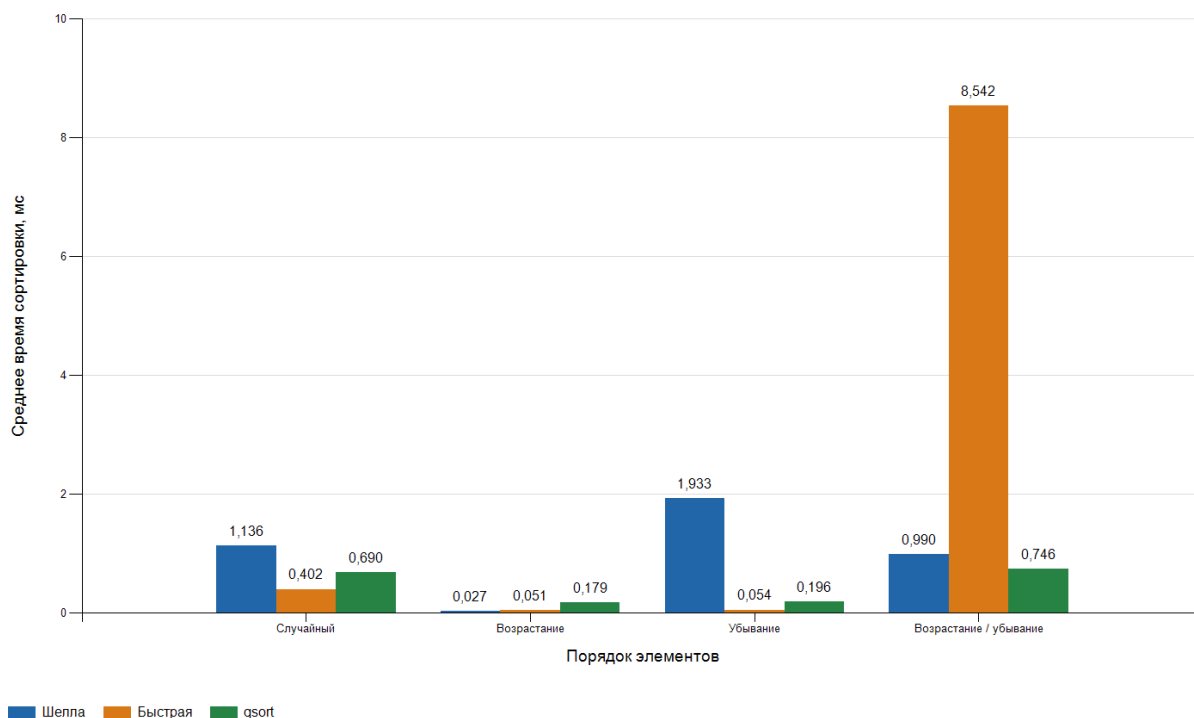


Рисунок 3 - Сравнение времени сортировок на четырёх типах данных

На случайном массиве минимальное время показала быстрая сортировка; на возрастающем - сортировка Шелла; на убывающем - быстрая сортировка. На смешанном массиве qsort оказалась быстрее двух других вариантов, а функция qs заметно замедлилась.

Выводы

В ходе лабораторной работы рассмотрены способы оценки времени выполнения программ на языке Си и исследована зависимость времени от размера и структуры входных данных.

Задание 1. Для стандартного алгоритма умножения двух квадратных матриц установлена сложность $\Theta(n^3)$ по времени и $\Theta(n^2)$ по памяти. Получены измерения для семи размеров от 100 до 10000. На больших матрицах основную часть полного времени занимает умножение.

Простая кубическая модель с коэффициентом, найденным при $n = 1000$, занижает время для больших размеров. Асимптотическая оценка описывает рост числа операций, но для прогноза времени необходимо учитывать особенности доступа к памяти, кэш и условия выполнения.

Задание 2. Сравнены сортировка Шелла, быстрая сортировка и qsort на четырёх типах массива из 10000 элементов. На случайных данных функция qs показала наименьшее время - около 0,402 мс. На возрастающих данных сортировка Шелла заняла около 0,027 мс.

Для массива с возрастающей и убывающей половинами время qs увеличилось примерно до 8,542 мс, тогда как qsort выполнила сортировку примерно за 0,746 мс. Результат показывает влияние исходного порядка и выбора опорного элемента на работу конкретной реализации.

Освоены измерение длительности отдельных участков программы, усреднение коротких запусков, сопоставление результатов разных алгоритмов и построение графиков по экспериментальным данным.